

Phạm Xuân Quý – B22DCAT240

Microservices Architecture Diagram



API Documentation

This document outlines the exposed REST API endpoints for the Bookstore Microservices project. The API requests are intended to be routed through the api-gateway which forwards them to the respective service.

Base URL

All API requests (other than direct gateway UI routes) are typically prefixed with the API gateway host and port.

Example: `http://localhost:8000/api/<service_name>/`

1. Gateway & Authentication Routes (api-gateway)

These endpoints are handled directly by the API Gateway to serve pages and manage global auth.

Method	Endpoint	Description
GET	/	Home page
GET	/auth/	Authentication view/page
POST	/login/	Process user login and retrieve tokens
GET	/book/<int:book_id>/	Gateway view for book details
GET	/cart/	Cart page view
GET	/checkout/	Checkout page view
GET	/orders/	User orders dashboard
GET	/staff/	Staff dashboard view
GET	/manager/	Manager dashboard view
GET	/api/docs/	Swagger UI documentation
GET	/api/redoc/	ReDoc API documentation

2. Book Service (book-service)

Manages the core book inventory, prices, authorship, and availability.

Method	Endpoint	Description
GET	/api/book-service/books/	Retrieve a list of all active books
POST	/api/book-service/books/	Create a new book record
GET	/api/book-service/books/<id>/	Retrieve specific book details by ID

Method	Endpoint	Description
PUT	/api/book-service/books/<id>/	Update an existing book's entirety
PATCH	/api/book-service/books/<id>/	Partially update an existing book
DELETE	/api/book-service/books/<id>/	Remove a book from inventory

3. Cart Service (cart-service)

Manages user shopping carts and active session items.

Method	Endpoint	Description
POST	/api/cart-service/carts/	Create a new unified cart
GET	/api/cart-service/carts/<customer_id>/	Retrieve the active cart for a customer
POST	/api/cart-service/cart-items/	Add a new item to the cart
PUT	/api/cart-service/cart-items/<item_id>/	Update a cart item (e.g., quantity)
DELETE	/api/cart-service/cart-items/<item_id>/	Remove an item from the cart

4. Order Service (order-service)

Handles order processing, fulfillment lifecycle, and checkout history.

Method	Endpoint	Description
GET	/api/order-service/orders/	List all orders (filtered by user context)
POST	/api/order-service/orders/	Create a new order (checkout)

Method	Endpoint	Description
PUT/PATCH	/api/order-service/orders/<id>/status/	Update the delivery/payment status of an order

5. Pay Service (pay-service)

Handles transactions, external payment intents, and methods.

Method	Endpoint	Description
GET	/api/pay-service/payment-methods/	List all available payment methods
POST	/api/pay-service/payments/	Process a new payment transaction

6. Catalog Service (catalog-service)

Manages product categorization and tags.

Method	Endpoint	Description
GET	/api/catalog-service/categories/	List all book categories / genres
POST	/api/catalog-service/categories/	Create a new category

7. Customer Service (customer-service)

Manages user and profile details.

Method	Endpoint	Description
GET	/api/customer-service/customers/	List customer profiles
POST	/api/customer-service/customers/	Register/Create a new customer profile

8. Ship Service (ship-service)

Manages shipping integration, delivery tracking, and providers.

Method	Endpoint	Description
GET	/api/ship-service/shipping-methods/	List supported shipping providers/methods
POST	/api/ship-service/shippings/	Create a new shipping manifest or update state

9. Comment & Rate Service (comment-rate-service)

Manages user-generated reviews, numerical ratings, and feedback on books.

Method	Endpoint	Description
GET	/api/comment-rate-service/reviews/	Fetch reviews (filterable by book ID)
POST	/api/comment-rate-service/reviews/	Submit a new review/rating

10. Recommender AI Service (recommender-ai-service)

Provides AI-driven suggestions.

Method	Endpoint	Description
POST/GET	/api/recommender-ai-service/ai-suggest/	Request AI-based book recommendations

11. Staff Service (staff-service)

Manages internal staff roles and privileged actions on books.

Method	Endpoint	Description
POST	/api/staff-service/staffs/	Register / Onboard new staff
POST/PUT	/api/staff-service/staff-books/	Advanced inventory and management queries for books

12. Manager Service (manager-service)

Manages high-level aggregation and corporate reporting.

Method	Endpoint	Description
GET	/api/manager-service/reports/	Generate and fetch sales, inventory, and activity reports

Technical Report: Bookstore Microservices System

Prepared for: Software Engineering Project / Microservice Architecture System

Overview: Scalable E-commerce Platform

Executive Summary

The Bookstore Microservices System is an enterprise-grade e-commerce application developed using a modern, scalable microservices architecture. Instead of relying on a traditional monolithic design, the system has been decoupled into twelve independent, highly cohesive, and loosely coupled services, alongside a dedicated API Gateway. This Technical Report provides a comprehensive overview of the architectural decisions, service-level responsibilities, database strategies, security protocols, and deployment environments that constitute the system.

The architecture leverages Django and Django REST Framework (DRF) as the underlying technology stack for individual services, ensuring rapid development, robust ORM capabilities, and straightforward API generation. The system operates primarily within a Dockerized environment, orchestrated by Docker Compose, to ensure seamless parity between development and production setups.

1. Introduction & Objectives

1.1 Project Purpose

The purpose of the Bookstore Microservices project is to simulate a real-world, high-traffic e-commerce platform capable of handling diverse operational loads. By splitting the application into microservices, the system achieves targeted scalability, fault isolation, and independent deployment cycles.

1.2 Core Objectives

- **Scalability:** Allow individual components (such as the order-service or recommender-ai-service) to scale horizontally without impacting the performance of other domains (like the staff-service).
- **Resilience:** Ensure that the failure of a single non-critical service (e.g., comment-rate-service) does not take down the entire purchasing pipeline.
- **Maintainability:** Enforce strict domain boundaries, preventing “spaghetti code” and allowing specialized teams to manage distinct services.
- **Technology Agnosticism (Potential):** While the current iteration relies heavily on Python/Django, the HTTP/REST boundary allows future services to be written in Rust, Go, or Node.js as needed.

2. High-Level System Architecture

The overarching architecture follows a pattern where a primary Gateway routes all external traffic to internal microservices.

2.1 The Monorepo Approach

The application is structured as a Monorepo, containing all independent service directories at the root level. This approach simplifies dependency management, shared testing strategies, and global Docker orchestration via a single docker-compose.yml file.

2.2 The API Gateway Pattern

In a microservices ecosystem, exposing all internal services directly to the public internet creates a massive attack surface and complicates front-end integration. To resolve this, the system implements an **API Gateway** (api-gateway running on port 8000). - **Reverse Proxying:** All API calls from the client (e.g., /api/order-service/orders/) are intercepted by the Gateway and securely proxied to internal network addresses (e.g., http://order-

service:8000/api/orders/). - **Frontend Delivery:** The Gateway also acts as a monolithic frontend server, rendering essential HTML templates and managing global session state. This hybrid approach (Server-Side Rendering + REST APIs) provides immediate SEO benefits while allowing dynamic interactions.

3. Microservices Domain Breakdown

The system is decomposed into 12 distinct domains, each bounded by specific business logic.

3.1 Book Service (book-service)

Role: Manages the core product catalog representing physical or digital books.

Responsibilities: - Tracking titles, authors, ISBNs, base pricing, and stock levels. - Responding to search and filter queries from the API Gateway. - Ensuring inventory decrement operations during order fulfillment.

3.2 Cart Service (cart-service)

Role: Manages ephemeral state related to user shopping sessions. **Responsibilities:** - Maintaining a list of CartItems mapped to a specific Cart identifying a user session. - Calculating sub-totals and applying temporary promotional rules before checkout. - Managing cart lifecycle (creation, item updates, clearing upon successful checkout).

3.3 Catalog Service (catalog-service)

Role: Manages high-level categorization, genres, and ontological mapping of books.

Responsibilities: - Maintaining hierarchical category trees (e.g., Fiction -> Sci-Fi -> Cyberpunk). - Allowing rapid retrieval of books based on broad category tags rather than specific title searches.

3.4 Order Service (order-service)

Role: The transactional heart of the e-commerce platform. **Responsibilities:** -

Transforming a Cart into an immutable Order record. - Tracking order lifecycle statuses (Pending -> Paid -> Shipped -> Delivered). - Integrating deeply with the Pay-Service and Ship-Service to orchestrate fulfillment.

3.5 Customer Service (customer-service)

Role: Manages User/Client profiles. **Responsibilities:** - Storing customer addresses, preferences, and aggregate lifetime value metrics. - Separating public identity from system-level User authentication records (which may reside at the gateway or auth layer).

3.6 Pay Service (pay-service)

Role: Interfacing with financial gateways. **Responsibilities:** - Managing Payment Methods (Credit Card, PayPal, On-Delivery). - Processing payment logic and generating transaction receipts. - Interacting with the Order-Service to confirm successful fund capture.

3.7 Ship Service (ship-service)

Role: Logistics and delivery management. **Responsibilities:** - Managing different shipping tiers (Standard, Express, International). - Generating tracking updates and dispatch manifests for completed orders.

3.8 Comment & Rate Service (comment-rate-service)

Role: User-generated content management. **Responsibilities:** - Storing written reviews and numerical 1-5 star ratings for specific books. - Calculating aggregated review scores asynchronously to avoid slowing down book-service queries.

3.9 Recommender AI Service (recommender-ai-service)

Role: Machine learning and analytics. **Responsibilities:** - Analyzing user purchase history and viewing habits to generate personalized “Books you might like” lists. - Serving pre-computed machine learning inferences via the /ai-suggest/ endpoints.

3.10 Staff & Manager Services (staff-service, manager-service)

Role: Internal administration and business intelligence. **Responsibilities:** - **Staff Service:** Allows bookstore employees to update inventory, process offline returns, and manage daily store operations. - **Manager Service:** Aggregates data from pay-service, order-service, and book-service to generate financial reports, sales charts, and high-level BI dashboards.

4. Data Layer and Persistence Strategy

4.1 Centralized PostgreSQL Database

While a strictly pure microservices architecture advocates for “database-per-service” to ensure absolute isolation, this project implements a **Centralized Database Pattern**

(default_db via PostgreSQL 15). - **Rationale:** For an academic or mid-tier enterprise project, orchestrating 12 separate database instances incurs massive memory overhead and introduces complex distributed transaction problems (e.g., Two-Phase Commit or Saga Patterns). - **Logical Isolation:** Despite residing in a single physical instance, the data layout is logically partitioned. Django's ORM ensures that each service only interacts with its own declared tables via migrations. Cross-service joins at the SQL level are strictly prohibited in the code; services must communicate via API to retrieve related data.

4.2 Data Integrity

By using PostgreSQL, the system benefits from advanced concurrency controls, robust ACID compliance, and sophisticated JSONB indexing, which is occasionally utilized by the AI and telemetry services for unstructured data storage.

5. Network Communication and Routing

The system relies strictly on synchronous HTTP/REST communication for real-time reads.

5.1 Universal API Proxying Protocol

To minimize the maintenance overhead of writing mapping logic for every new endpoint, the api-gateway employs a "Universal Proxy" view. A path such as: `http://localhost:8000/api/order-service/orders/123/` Is dynamically intercepted by the Gateway. The Gateway extracts the service_name (order-service) and the remaining path (/orders/123/). Using Python's requests library, the Gateway acts as a reverse proxy, forwarding the client's payload (Headers, GET params, POST body) directly to the internal Docker network alias `http://order-service:8000/orders/123/`.

5.2 Internal Service Service Discovery

Because the services are deployed via Docker Compose, Docker's internal DNS handles service discovery. order-service will resolve to the internal IP of the respective container, regardless of host machine IP changes.

6. Authentication and Security

6.1 Authentication Flow

Authentication is managed centrally to prevent users from having to authenticate 12 different times. - Users POST to /login/ on the Gateway. - The Gateway verifies credentials (often against a local shared User table or dedicated Identity service). - Sessions or JWT (JSON Web Tokens) are established. The Gateway attaches validation headers to all downstream proxy requests, allowing internal microservices to trust the identity of the incoming request without hitting the database repeatedly.

6.2 Service-to-Service Security

Currently, services within the Docker network (172.x.x.x) trust traffic implicitly if it originates from the Gateway. Direct external access to nodes (e.g., hitting book-service on port 8002 directly) is intended for local debugging only. In a production environment, only port 8000 (Gateway) is exposed to the host firewall.

7. Containerization and Infrastructure

7.1 Docker Compose Configuration

The docker-compose.yml serves as the Infrastructure-as-Code (IaC) definition for the platform. - **Base Images:** All Python services are containerized using lightweight Python Alpine or Slim images, significantly reducing the surface area for vulnerabilities. - **Volume Mounting:** During development, local directories (./book-service:/app) are mapped to the containers. This enables “hot-reloading,” allowing developers to edit views.py on their host machine and instantly see changes inside the container without rebuilding the Docker image. - **Database Initialization:** A volume map ./postgres-init:/docker-entrypoint-initdb.d is utilized to pre-seed the database schema and root users automatically when the cluster starts for the very first time.

7.2 Port Mapping Strategy

To avoid conflicts, port bindings increment sequentially: - Gateway: 8000 - Customer: 8001 - Book: 8002 - Cart: 8003 - Staff: 8004 - Manager: 8005 - Catalog: 8006 - Order: 8007 - Ship: 8008 - Pay: 8009 - Comment/Rate: 8010 - Recommender-AI: 8011

8. Automated API Documentation (Swagger/OpenAPI)

Documentation is critical for a decoupled system. The project integrates drf-spectacular to automatically generate OpenAPI 3.0 schemas. - **Dynamic Schema:** The generation occurs directly from DRF serializers and viewsets, meaning the documentation is strictly

ties to the code and cannot fall out of date. - **UI Interfaces:** Both Swagger UI (/api/docs/) and ReDoc (/api/redoc/) are served by the Gateway, providing interactive sandboxes where developers and QA testers can safely mock API requests against the live system.

9. Future Work & Scalability Prospects

While robust, the current architecture has several avenues for future optimization: 1. **Asynchronous Message Brokers (RabbitMQ/Kafka):** Moving from synchronous HTTP REST calls between services (e.g., Order -> Pay) to asynchronous event-driven queues (e.g., OrderCreated event) would massively improve resilience. If the Pay service is down, the message stays in the queue until the service recovers. 2. **Caching Layers (Redis/Memcached):** The catalog-service and book-service are heavily read-optimized. Placing a Redis cache in front of these services would reduce PostgreSQL load by 90% during peak traffic. 3. **Database-Per-Service Migration:** As data scales, migrating from default_db to isolated physical data stores (or migrating specific high-transaction services like order-service to a NoSQL store like MongoDB) would increase throughput.

10. Conclusion

The Bookstore Microservices platform effectively demonstrates the power and flexibility of modernized backend engineering. By decoupling the monolithic constraints of traditional Django apps into 12 tightly focused domains, the system offers unparalleled maintainability and scalability. The presence of the API Gateway seamlessly unifies the fragmented backend for the client applications, while Docker ensures that the complex orchestration of 13 separate containers occurs flawlessly across varying operating systems. The implementation forms a robust foundation for a production-ready, globally scalable enterprise application.

WEB UI



Hệ thống quản trị Microservices

Tài khoản



Mật khẩu



ĐĂNG NHẬP ↗

Chưa có tài khoản? [Đăng ký ngay](#)

Tài khoản Demo:

`admin`: Trang Nhân viên | `manager`: Trang Sếp

Khác: Khách hàng đi chợ



Đăng ký tài khoản mới

Họ và Tên



Tên đăng nhập



Mật khẩu



ĐĂNG KÝ 👤

Đã có tài khoản? [Quay lại Đăng nhập](#)

Q Tìm kiếm

Thế loại

Khoảng giá (đ)

Sắp xếp

A Tên sách, tác giả...

Tất cả thể loại

Từ giá...

- Đến giá...

Mặc định

Tất Cả Sản Phẩm

28 sản phẩm



Leo Tolstoy
War and Peace

Chiến Tranh và Hòa Bình (...)

Leo Tolstoy

175000đ

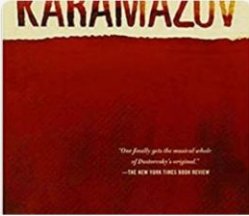


FYODOR DOSTOEVSKY
Crime and Punishment

Tội Ác và Hình Phạt (Crim...)

Fyodor Dostoevsky

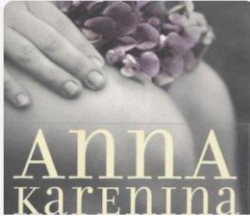
125000đ



Anh Em Nhà Karamazov (...)

Fyodor Dostoevsky

155000đ



Anna Karenina

Leo Tolstoy

145000đ

Thêm vào giỏ

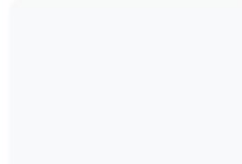
Thêm vào giỏ

Thêm vào giỏ

Thêm vào giỏ

Gợi Ý Hôm Nay

Dựa trên thể loại bạn yêu thích: Sách mới đang bán cực chạy, xả kho giá hời!



AI Superpowers - Kỳ Nguyên AI

128000đ

Thêm vào giỏ



The Pragmatic Programmer

175000đ

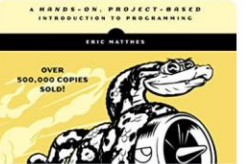
Thêm vào giỏ



Design Patterns - Gang of Four

195000đ

Thêm vào giỏ

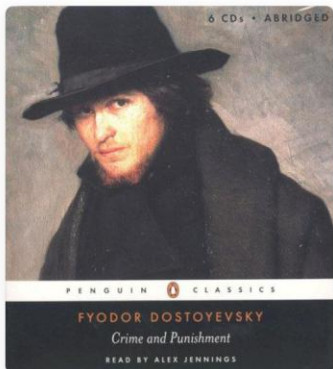


Python Crash Course

195000đ

Thêm vào giỏ

← Quay lại Cửa hàng



Tội Ác và Hình Phạt (Crime and Punishment)

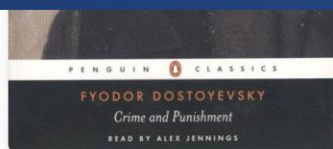
Tác giả: Fyodor Dostoevsky

125000đ

Kho còn: 60 cuốn

1

Thêm Vào Giỏ Ngay



Đánh giá từ khách hàng

Khách hàng #8

Tội Ác và Hình Phạt - Đọc âm ảnh và rùng mình về tâm lý nhân vật

5 ★

Viết đánh giá của bạn

Chăm điểm (1-5 sao):

★★★★★ (5 Sao)

Bình luận:

Sách này đọc có bảnh cuốn không?

Gửi Đánh Giá

Giỏ Hàng Của Bạn

Sản phẩm	Đơn giá	Số lượng	Thành tiền	Thao tác
 Tội Ác và Hình Phạt (Crime and Punishment)	125000đ	1	125000đ	<button>Xóa</button>

Tóm tắt đơn hàng

Tổng cộng: **125000đ**

[Tiến Hành Thanh Toán](#) →

QuyStore | Chốt Đơn

1. Địa chỉ giao hàng

Thái Bình

2. Phương thức vận chuyển

☒ **Giao hàng tiêu chuẩn**

Nhận hàng trong 3-5 ngày

+2đ

☐ **Giao hàng Hỏa tốc**

Nhận hàng trong 24h

+5đ

3. Phương thức thanh toán

☒ **Thanh toán khi nhận hàng (COD)**

☐ Chuyển khoản / Thẻ tín dụng

Tóm tắt đơn hàng

Tội Ác và Hình Phạt (Crime and Punishment)
Số lượng: 1

125000đ

Tổng tiền sách:

125000đ

Phí giao hàng:

+2.00

Tổng cộng:

\$125002.00

[ĐẶT HÀNG](#)

QuyStore

Cửa hàngAn.nguyen

Lịch Sử Đơn Hàng

← Tiếp tục mua sắm

Mã đơn: #1

Đang giao hàng

Địa chỉ nhận hàng:
Thái Bình

Tổng thành tiền
125002đ

Staff Portal

QuyStore Management

NGHIỆP VỤ

Quản lý Đơn hàng

Quản lý Kho sách

Staff Admin

Nhân viên

Đăng xuất

Bảng điều khiển Nhân viên

Quản lý đơn hàng và kho sách của QuyStore

TỔNG ĐƠN HÀNG

TỔNG ĐƠN CHỜ XỬ LÝ

TỔNG ĐẦU SÁCH

TỔNG TỐN KHO

Danh sách Đơn hàng

Tổng 1 đơn

Tất cảChờ xử lýĐã duyệtĐang giaoHoàn thànhĐã hủy

MÃ ĐƠN	KHÁCH HÀNG	TỔNG TIỀN	ĐỊA CHỈ GIAO	NGÀY TẠO	TRẠNG THÁI	THAO TÁC
#1	1 KH_1	125002đ	Thái Bình	2026-03-14T08:10:46.821596Z	Đang giao	✓ Duyệt ✕

Staff Portal

QuyStore Management

NGHIỆP VỤ

Quản lý Đơn hàng

Quản lý Kho sách

Bảng điều khiển Nhân viên

Quản lý đơn hàng và kho sách của QuyStore

1

TỔNG ĐƠN HÀNG

0

ĐƠN CHỜ XỬ LÝ

28

TỔNG ĐẦU SÁCH

2.813

TỔNG TỒN KHO

Kho sách hiện tại

Nhập sách mới

ID	ẢNH BÌA	TÊN SÁCH	TÁC GIẢ	GIÁ BÁN	TỒN KHO	DANH MỤC	THAO TÁC
#28		Truyện Cổ Grimm (Fairy Tales by Brothers Grimm)	Anh Em Nhà Grimm	75000đ	180	Thiếu nhi	Edit Delete
#27		Hoàng Tử Bé (The Little Prince)	Antoine de Saint-Exupéry	65000đ	250	Thiếu nhi	Edit Delete
#26		Homo Deus: Lược Sử Tương Lai	Yuval Noah Harari	155000đ	95	Lịch sử - Địa lý	Edit Delete

Manager HQ

QuyStore Analytics

BÁO CÁO

Tổng quan KPI

Nhân viên & Kho

Cửa hàng

M Manager

Cấp quản lý

Đăng xuất

Bảng điều khiển KPI

Dữ liệu cập nhật từ hệ thống — QuyStore Analytics

125002đ

TỔNG DOANH THU

Cộng dồn từ 1 đơn hàng

1

TỔNG ĐƠN HÀNG

0 đã giao thành công

0

ĐƠN TỒN ĐÓNG

Cần nhân viên xử lý gấp!

0

ĐƠN HOÀN THÀNH

0 đã bị hủy

Phân tích Trạng thái

1 đơn

Đang giao

1 (100%)

Sao kê Giao dịch gần nhất

10 đơn mới nhất

MÃ ĐƠN	KHÁCH HÀNG	DOANH THU	PHƯƠNG THỨC	TRẠNG THÁI
#1	1 KH_1	125002đ	COD	Đang giao